Московский государственный университет

имени М. В. Ломоносова

Майстренко Марат Алексеевич

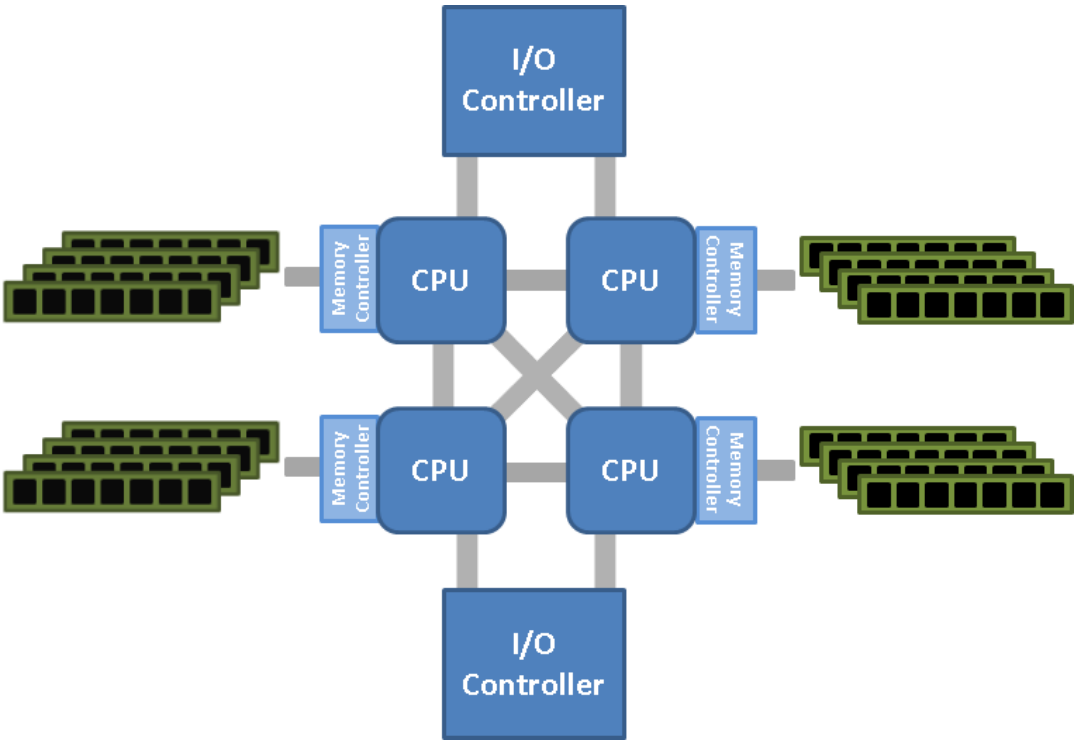
«Раскрытие проблемы неподходящей локальности использования памяти»

Москва, 2025

1.Постановка проблемы

В современных параллельных вычислениях эффективная организация доступа к данным является определяющим фактором производительности. Одним из ключевых инструментов для организации таких вычислений является MPI (Message Passing Interface), который предоставляет богатый набор функций для обмена данными, включая прямой доступ к памяти удалённых процессов (Remote Memory Access, RMA). Однако неправильное использование этих возможностей может привести к серьёзным потерям производительности.

В частности, проблема возникает, когда все процессы многократно запрашивают одни и те же данные с одного главного процесса вместо того, чтобы заранее распределить данные между собой. Такой подход (централизованный доступ) создаёт «узкое место» на главном процессе, приводя к очередям запросов, росту задержек и снижению общей масштабируемости. Наоборот, оптимальная схема — это когда каждый процесс забирает свою часть данных заранее и производит локальные вычисления, после чего происходит минимальный обмен итоговыми результатами. Такой подход позволяет существенно снизить сетевые накладные расходы и добиться близкой к линейной масштабируемости даже при большом числе процессов.



Обращение в свою память – доли секунды, при этом обращение в память другого вычислительного узла – довольно затратная задача, порождающая новые накладные расходы и создающая узкое место в программе.

2.Пример неправильного кода

Рассмотрим задачу вычисления суммы элементов массива, которая хорошо поддаётся распараллеливанию: массив делится между P процессами, каждый процесс подсчитывает свою часть, а затем формируется общая сумма.

Однако в неоптимальной реализации **каждый процесс многократно запрашивает элементы массива с главного (нулевого) процесса во время вычислений**. Для этого используется механизм MPI RMA, где процессы через MPI_Get читают данные напрямую из удалённого окна памяти.

Логически это понятно: все процессы получают нужные данные, выполняют подсчёты параллельно. Но на практике это создаёт узкое место — главный процесс становится центром обработки запросов. Каждый MPI_Get требует синхронизации через MPI_Win_fence, что параллельную программу превращает в последовательную, ведь все обращения к памяти централизованы.

Основная проблема — **централизованный доступ всех процессов к общим данным** во время вычислений.

В программе это реализовано так:

1. Каждый процесс многократно вызывает MPI_Get, запрашивая элемент
2. После суммирования все процессы выполняют MPI_Accumulate

Последствия:

- **Накладные расходы:** главный процесс вынужден обслуживать огромное количество мелких запросов, что ведёт к перегрузке сети и синхронизаций.
- **Снижение производительности:** вместо локальных быстрых вычислений нагрузка перекладывается на коммуникацию, которая доминирует во времени выполнения.

```
void bad_access(int N, int rank, int size, MPI_Win win, int* shared_array) {
    int local_sum = 0;

    MPI_Win_fence(0, win);

    for (int i = rank; i < N; i += size) {
        int value;
        MPI_Get(&value, 1, MPI_INT, 0, i, 1, MPI_INT, win);
        local_sum += value;
    }

    MPI_Win_fence(0, win);
    MPI_Accumulate(&local_sum, 1, MPI_INT, 0, N, 1, MPI_INT, MPI_SUM, win);
    MPI_Win_fence(0, win);
}
```

3. Исправление неправильного кода

Чтобы устранить проблему централизованного доступа, нужно **заранее распределить данные** между процессами. Это позволяет организовать параллельную работу, где каждая вычислительная единица работает независимо, без необходимости постоянной синхронизации с главным процессом

```
void good_access(int N, int rank, int size, MPI_Win win, int* shared_array) {
    int block_size = N / size;
    int start = rank * block_size;
    int end = (rank == size - 1) ? N : start + block_size;
    int my_count = end - start;
    std::vector<int> local_block(my_count);

    MPI_Win_fence(0, win);
    MPI_Get(local_block.data(), my_count, MPI_INT, 0, start, my_count, MPI_INT, win);
    MPI_Win_fence(0, win);

    int local_sum = 0;
    for (int v : local_block)
        local_sum += v;

    MPI_Win_fence(0, win);
    MPI_Accumulate(&local_sum, 1, MPI_INT, 0, N, 1, MPI_INT, MPI_SUM, win);
    MPI_Win_fence(0, win);
}
```

4. Измерение ускорения и анализ результатов

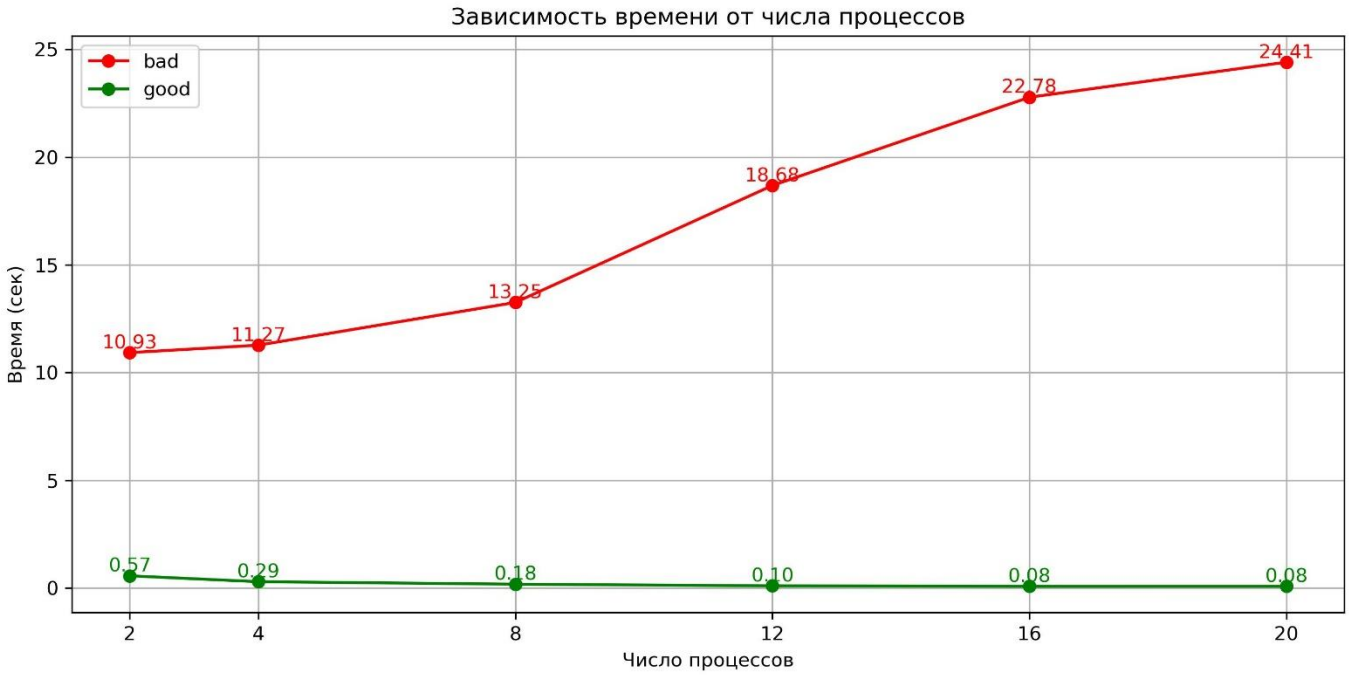
Процессы	Bad_access [c.]	Good_access [c.]
2	10.93	0.57
4	11.27	0.29
8	13.25	0.18
12	18.68	0.10
16	22.78	0.08
20	24.41	0.08

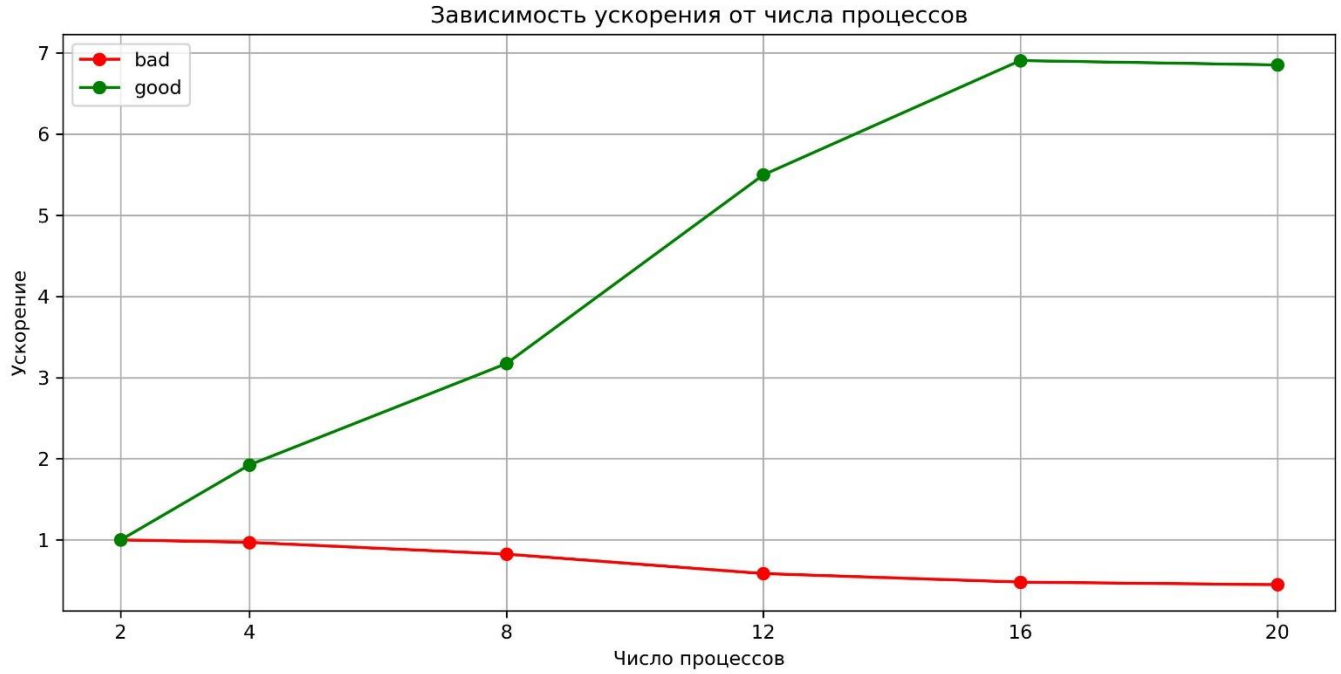
good_access (оптимизированная версия):

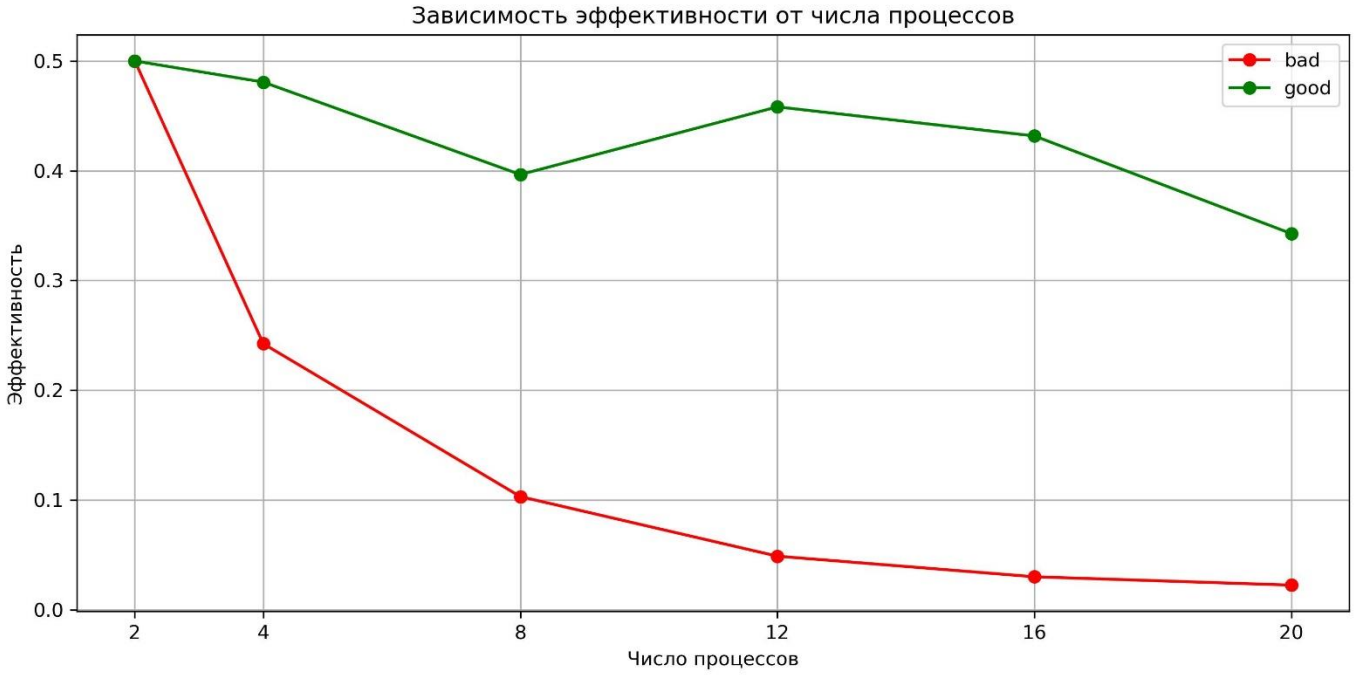
- Показывает отличную масштабируемость: с увеличением числа процессов время стабильно снижается.
- При 20 процессах достигается **максимальное ускорение** (~0.08 сек), что почти **300-кратное** ускорение по сравнению с плохой версией

bad_access (неоптимизированная версия):

- Масштабируемость почти отсутствует: с ростом числа процессов время даже **увеличивается**, а не уменьшается.
- Накладные расходы на синхронизацию полностью поглощают выигрыш от параллельных вычислений.


Зависимость времени от числа процессов
bad
good
Время (сек)
Число процессов
10.93
11.27
13.25
18.68
22.78
24.41
0.57
0.29
0.18
0.10
0.08
0.08


Зависимость ускорения от числа процессов
bad
good
Ускорение
Число процессов


Зависимость эффективности от числа процессов
bad
good
Эффективность
Число процессов

5. Профилирование программы с помощью mpiP

Программа имеет высокую зависимость от MPI-коммуникаций: среднее время MPI составляет *98.02%* общего времени выполнения (487 из 496 секунд).

Основными узкими местами являются:

- Win_fence (51.65% времени) — синхронизационные барьеры, связанные с оконными операциями. Частое ожидание завершения операций RMA.
- Get (46.68% времени) — массовые запросы данных (50 млн. вызовов), крайне частые обращения к удалённой памяти

Программа явно страдает от **частого доступа к удалённым данным и избыточной синхронизации**, что является типичным для неэффективно спроектированных схем с RMA.

```
---------------------------------------------------------------------------
@--- Aggregate Time (top twenty, descending, milliseconds) ----------------
---------------------------------------------------------------------------
Call                 Site       Time    App%    MPI%      Count    COV
Win_fence               1   2.51e+05   50.63   51.65         20   0.92
Get                    14   2.27e+05   45.75   46.68   50000000   1.02
Win_create              3   7.52e+03    1.51    1.55         20   0.23
Win_fence              10        400    0.08    0.08         20   0.34
Win_fence              15        127    0.03    0.03         20   0.34
Get                     9       42.1    0.01    0.01         20   1.03
Win_free               16       21.5    0.00    0.00         20   0.20
Win_fence               8       2.23    0.00    0.00         20   0.49
Accumulate              6       1.53    0.00    0.00         20   0.42
Win_fence               5       1.48    0.00    0.00         20   0.59
Win_fence              12       1.42    0.00    0.00         20   0.22
Barrier                 4      0.778    0.00    0.00         20   0.33
Barrier                17      0.607    0.00    0.00         20   0.22
Win_fence              11      0.315    0.00    0.00         20   0.14
Accumulate             13       0.26    0.00    0.00         20   1.23
Barrier                 7      0.201    0.00    0.00         20   0.25
Barrier                 2      0.148    0.00    0.00         20   0.37
```

6. Заключение

Исследование наглядно показывает, что даже при хорошем балансе нагрузки эффективность может ограничиваться (bad access) плохой локальностью данных, что приводит к лишним накладным расходам. При этом грамотно локализованный и упорядоченный обмен (good access) позволяет значительно сократить как количество, так и стоимость коммуникаций, снижая общие накладные расходы.

Это подчёркивает важность тщательного проектирования стратегии доступа и обмена данными, минимизируя дорогие синхронизации и использования затратных средств MPI средств.